Evolutionary Computation Assignment 4: Candidate Moves

BENGRICHE Tidiane (ER-2037)
Szczublińska Joanna Wiktoria (156070)

Contents

1 Introduction

The purpose of this report is to **improve the time efficiency of the steepest local search algorithm** by using **candidate moves** to leveraging the neighborhood structure which was the most effective in the previous assignment, but its computational cost was quite high when applied to our instances. By introducing candidate moves, we aim to significantly reduce the number of evaluated moves at each iteration while maintaining high solution quality.

Candidate moves are defined as moves that introduce at least one candidate edge into the solution. Candidate edges are determined for each vertex by selecting a fixed number of "nearest" vertices, where "nearest" is defined based on a combination of edge length and vertex cost. In this work, we use 10 nearest vertices per node as candidate edges for default, although this parameter will be adjusted experimentally by us.

The report explores both **intra-route** and **inter-route moves**. In intra-route moves, such as two-edge exchanges, candidate edges ensure that at least one promising edge is added to the solution. In inter-route moves, which involve exchanging selected and unselected nodes, it is similarly required that at least one candidate edge is introduced. This ensures that the algorithm focuses on moves that are more likely to improve the solution rather than performing random or unproductive exchanges.

Random solutions are used as starting paths for the local search. For comparison, we also report results of the standard steepest local search with random starting solutions, without the use of candidate moves. This allows us to measure the improvements in both computational time and solution quality achieved by the candidate move strategy.

In the following sections, we describe the implementation of the local steepest search with candidate moves, discuss the impact of varying the number of candidate edges, and present experimental results demonstrating the trade-off between computation time and solution quality. Finally, we analyze the effectiveness of candidate moves and provide conclusions on their usefulness for improving local search algorithms in combinatorial optimization problems.

You can see the repository of our assignment on github:
`https://github.com/tbengric/EC_laboratory`

2 Pseudocode

In this section, we will present the pseudocodes of the algorithms. For this assignment, we will code in C++.

Candidate Moves

```
candidateMoves(nodes, distMatrix, K) :
    n ← LENGTH(nodes)
    candidateNeighbors ← ARRAY of n empty lists
    FOR i FROM 0 TO n-1 DO :
        neighborCosts ← empty list
        FOR j FROM 0 TO n-1 DO :
            IF i = j THEN CONTINUE
            cost ← distMatrix[i][j] + nodes[j].cost
            APPEND (cost, j) TO neighborCosts
        END FOR
        SORT neighborCosts BY ascending cost
        candidateNeighbors[i] ← first K indices from neighborCosts
    END FOR
    RETURN candidateNeighbors
```

In our assignment we experiment with different values of k to observe how the change will influence the objective value. The default value of k for function is 10

Local Search Candidate (Part 1)

```
steepestLocalSearchCandidates(initialPath, distMatrix, nodes, K) :
    currentPath ← COPY(initialPath)
    pathSize ← LENGTH(currentPath)
    n ← LENGTH(nodes)
    candidateNeighbors ← candidateMoves(nodes, distMatrix, K)
    pos ← ARRAY of n elements (-1)
```

Local Search Candidate (Part 2)

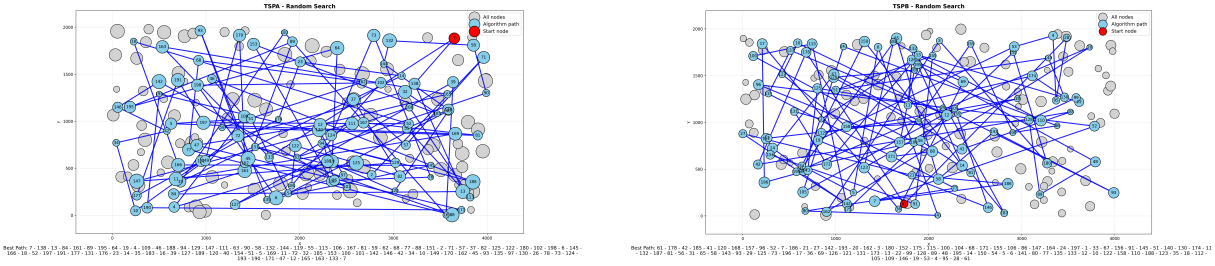
```
WHILE TRUE DO :
  bestDelta ← 0
  bestMoveType ← NONE
  bestFirst ← -1, bestSecond ← -1, bestNewNode ← -1
  % Update positions of nodes
  FOR i FROM 0 TO pathSize-1 DO : pos[currentPath[i]] ← i END FOR
  % Examine candidate moves for each node
  FOR i FROM 0 TO pathSize-1 DO :
    nodeID ← currentPath[i]
    FOR neigh IN candidateNeighbors[nodeID] DO :
      % 1. Intra-route move
      j ← pos[neigh]
      IF j > 0 AND j < pathSize-1 AND ABS(i - j) > 1 THEN :
        delta ← deltaIntraEdge(currentPath, i, j, distMatrix)
        IF delta < bestDelta THEN : bestDelta ← delta, bestMoveType ← 1,
bestFirst ← i, bestSecond ← j END IF
      END IF
      % 2. Inter-route move
      IF pos[neigh] = -1 THEN :
        IF i-1 >= 0 THEN : delta ← deltaInterNode(currentPath, i-1, neigh,
distMatrix, nodes)
          IF delta < bestDelta THEN : bestDelta ← delta, bestMoveType ← 2,
bestFirst ← i-1, bestNewNode ← neigh END IF END IF
        IF i+1 < pathSize THEN : delta ← deltaInterNode(currentPath, i+1,
neigh, distMatrix, nodes)
          IF delta < bestDelta THEN : bestDelta ← delta, bestMoveType ← 2,
bestFirst ← i+1, bestNewNode ← neigh END IF END IF
      END IF
    END FOR
  END FOR
  % Apply best move
  IF bestMoveType = 1 THEN :
    a ← MIN(bestFirst, bestSecond), b ← MAX(bestFirst, bestSecond)
    REVERSE currentPath[a+1 TO b]
    FOR t FROM a+1 TO b DO pos[currentPath[t]] ← t END FOR
  ELSE IF bestMoveType = 2 THEN :
    oldNode ← currentPath[bestFirst], currentPath[bestFirst] ← bestNewNode
    pos[oldNode] ← -1, pos[bestNewNode] ← bestFirst
  ELSE : BREAK % no improvements
  END IF
  END WHILE
  RETURN currentPath
```

3 Plots

After trying these differents algorithms, we will check their effectiveness by showing the constructed cycle.

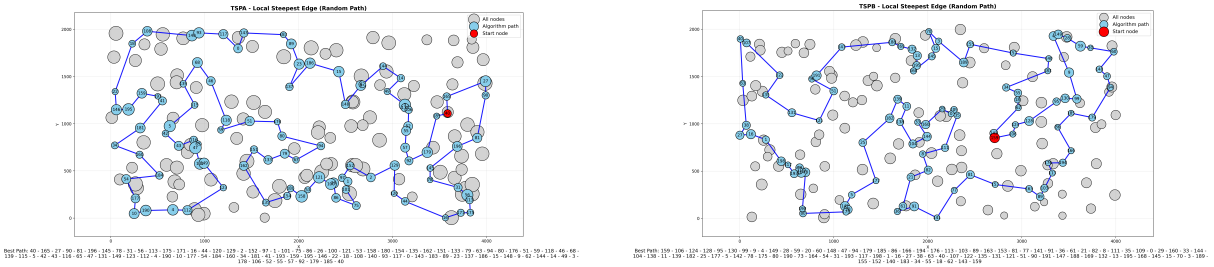
In the plot, the cost of each node is represented by its size: larger nodes indicate a higher cost, while smaller ones indicate a lower cost. Additionally, all possible nodes are displayed, but the nodes chosen for the path are shown in blue, with their IDs printed on them. The started node is pointed with red color.

Figures for Random Search and Steepest Local Search for Random Solution



(a) TSPA (b) TSPB

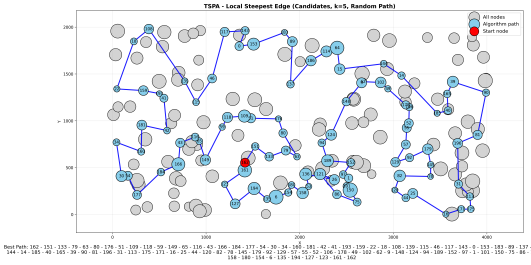
Figure 1: Random Search



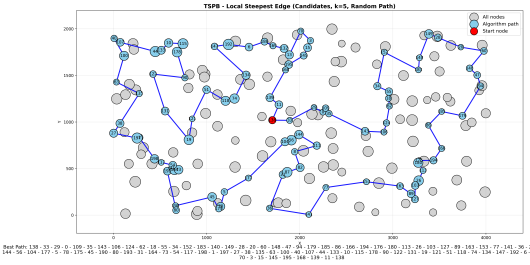
(a) TSPA (b) TSPB

Figure 2: Local Search for Random Path

Figures for Steepest Local Search with Candidate Moves

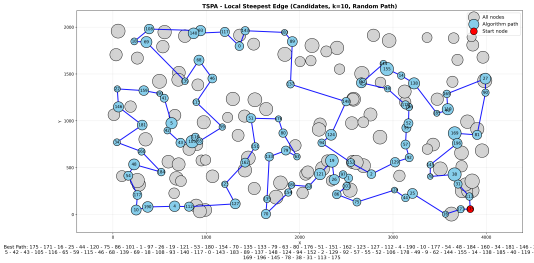


(a) TSPA

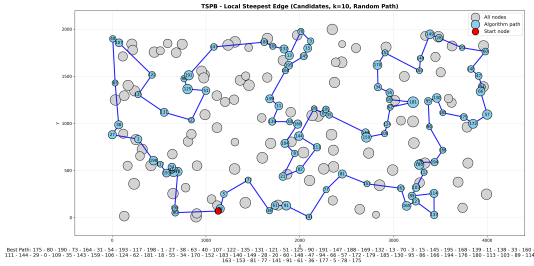


(b) TSPB

Figure 3: Local Search for Random Path with Candidates k=5

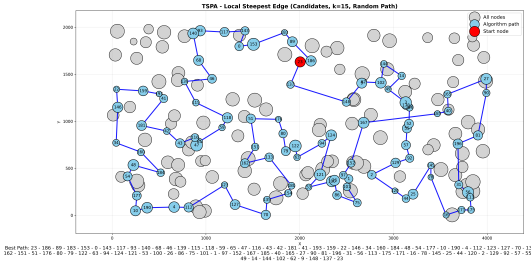


(a) TSPA

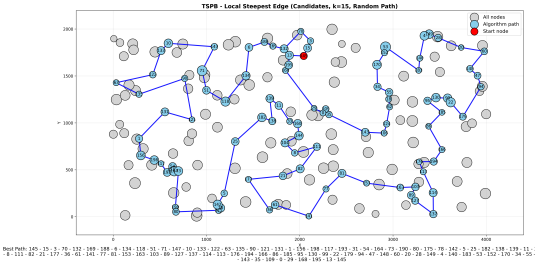


(b) TSPB

Figure 4: Local Search for Random Path with Candidates k=10



(a) TSPA



(b) TSPB

Figure 5: Local Search for Random Path with Candidates k=15

4 Results

Method	TSPA Avg (Min, Max)	TSPB Avg (Min, Max)
Random Search	263556.62 (233856, 291265)	211945.96 (185122, 239221)
Local Steepest (Random Path)	74288.36 (71159, 79944)	48517.03 (46105, 52225)
Local Steepest (Candidates k=5)	86097.13 (77488, 94234)	50656.25 (47796, 54729)
Local Steepest (Candidates k=10)	78063.28 (73894, 84469)	48582.53 (45605, 51705)
Local Steepest (Candidates k=15)	75737.19 (71916, 80604)	48397.54 (45792, 51581)

Table 1: Comparison of average, minimum, and maximum objective values for TSPA and TSPB.

Method	TSPA Time (avg, min, max)	TSPB Time (avg, min, max)
Random Search	0.0041 (0.0034, 0.0287)	0.0054 (0.0046, 0.0239)
Local Steepest (Random Path)	9.9808 (8.1813, 22.7102)	10.7825 (8.5106, 18.1593)
Local Steepest (Candidates k=5)	1.4362 (1.3117, 5.7258)	1.5195 (1.3399, 2.0822)
Local Steepest (Candidates k=10)	1.7788 (1.5927, 5.8011)	1.9997 (1.6765, 3.3668)
Local Steepest (Candidates k=15)	2.1685 (1.9311, 5.5743)	2.4723 (2.0156, 3.6432)

Table 2: Comparison of average, minimum, and maximum execution times in microseconds for TSPA and TSPB.

5 Conclusion

Based on the results in Tables 1 and 2, the following observations can be made:

The basic Local Steepest Edge algorithm provides one of the best results among all algorithms from previous assignments, although it can be time consuming, especially when using the Greedy Cycle as the starting path. **Introducing candidate lists significantly reduces computation time**, while still yielding relatively high objective values. In this case, it reduces computation time considerably and produces results quite similar to those obtained from the Nearest Neighbor method in the first assignment, but with much smaller time.

Introducing k candidates saves a lot of time and produces good results,as we have seen during different experiments with various values of k. We found that **larger k values generally give slightly better results with only a minor increase in computation time**. As we can see the **Candidate version is much better as we get very similar results very fast.** In our case the Steepest Local Search with Candidates k=15 gave even better results for TSPB then the basic Steepest version.

Using candidate lists saves time because we do not evaluate all nodes in each step. Instead, we check a fixed number of candidates, which drastically reduces the number of computations. Additionally, it is logical that **with a higher number of candidates**, we can **explore more** moves, which slightly **improves the objective value**.

6 All Results

Method	TSPA Avg (Min, Max)	TSPB Avg (Min, Max)
Random Search	263556.62 (233856, 291265)	211945.96 (185122, 239221)
Nearest Neighbor (End)	85108.51 (83182, 89433)	54390.43 (52319, 59030)
Nearest Neighbor (Flexible)	73594.10 (71868, 75953)	48694.25 (44609, 57283)
Greedy Cycle	72635.98 (71488, 74410)	51400.60 (49001, 57324)
Greedy 2-Regret	115400.24 (105692, 126860)	72712.85 (67809, 78406)
Greedy 2-Regret (w=0.5)	72136.15 (71108, 73395)	50934.60 (47144, 55700)
NN-Flex 2-Regret	117138.49 (108151, 124921)	74444.46 (69933, 80278)
NN-Flex 2-Regret (w=0.5)	72407.59 (70010, 75452)	47664.46 (44891, 55247)
Local Greedy Node (Greedy Cycle)	**70687.00 (70687, 70687)**	**46443.67** (46328, 51512)
Local Greedy Edge (Greedy Cycle)	**70618.84** (70571, 70663)	**46089.89** (45824, 51482)
Local Steepest Node (Greedy Cycle)	**70687.00 (70687, 70687)**	46466.54 (46371, 51512)
Local Steepest Edge (Greedy Cycle)	**70571.00** (70571, 70571)	**45996.84 (45867, 51259)**
Local Greedy Node (Random Path)	85217.85 (77570, 94277)	60178.87 (54006, 66885)
Local Greedy Edge (Random Path)	74232.03 (71777, 77750)	48192.19 (46109, 51934)
Local Steepest Node (Random Path)	85271.00 (85271, 85271)	63491.00 (63491, 63491)
Local Steepest Edge (Random Path)	74288.36 (71159, 79944)	48517.03 (46105, 52225)
Local Steepest Edge (Candidates, k=5, Random Path)	86097.13 (77488, 94234)	50656.25 (47796, 54729)
Local Steepest Edge (Candidates, k=10, Random Path)	78063.28 (73894, 84469)	48582.53 (45605, 51705)
Local Steepest Edge (Candidates, k=15, Random Path)	75737.19 (71916, 80604)	48397.54 (45792, 51581)

Table 3: Comparison of average, minimum, and maximum objective values for TSPA and TSPB. The 3 lowest average values in each column are highlighted in bold.

Method	TSPA Time Avg (Min, Max)	TSPB Time Avg(Min, Max)
Random Search	0.0041 (0.0034, 0.0287)	0.0054 (0.0046, 0.0239)
Nearest Neighbor (End)	0.0694 (0.0463, 0.1093)	0.0685 (0.0470, 0.1282)
Nearest Neighbor (Flexible)	91.0993 (85.8761, 119.1184)	94.7285 (87.4900, 154.2259)
Greedy Cycle	87.6467 (82.1853, 135.8448)	90.6419 (84.6333, 153.7690)
Greedy 2-Regret	13.4090 (12.2387, 18.2542)	13.4612 (12.4574, 21.9434)
Greedy 2-Regret (w=0.5)	12.9063 (11.8320, 18.2641)	12.6926 (11.2819, 20.7204)
NN-Flex 2-Regret	13.8750 (12.7669, 18.4833)	13.9626 (12.9940, 21.1062)
NN-Flex 2-Regret (w=0.5)	13.5540 (12.4458, 19.2731)	13.6345 (12.6257, 21.8523)
Local Greedy Node (Greedy Cycle*)	1.2763 (0.8682, 2.4062)	1.8935 (1.1700, 5.2516)
Local Greedy Edge (Greedy Cycle*)	1.5676 (1.1047, 3.2752)	2.5081 (1.3700, 5.8678)
Local Steepest Node (Greedy Cycle*)	0.5265 (0.4719, 1.0174)	0.7846 (0.4909, 1.6765)
Local Steepest Edge (Greedy Cycle*)	0.5418 (0.4837, 1.0700)	1.0705 (0.6829, 2.2377)
Local Greedy Node (Random Path)	57.7227 (44.0444, 123.7979)	51.7444 (41.5894, 92.8947)
Local Greedy Edge (Random Path)	58.0691 (47.6349, 99.7285)	52.4021 (45.1037, 84.4763)
Local Steepest Node (Random Path)	10.9774 (10.0351, 18.7647)	13.6565 (12.4567, 26.2057)
Local Steepest Edge (Random Path)	9.9808 (8.1813, 22.7102)	10.7825 (8.5106, 18.1593)
Local Steepest Edge (Candidates, k=5, Random Path)	1.4362 (1.3117, 5.7258)	1.5195 (1.3399, 2.0822)
Local Steepest Edge (Candidates, k=10, Random Path)	1.7788 (1.5927, 5.8011)	1.9997 (1.6765, 3.3668)
Local Steepest Edge (Candidates, k=15, Random Path)	2.1685 (1.9311, 5.5743)	2.4723 (2.0156, 3.6432)

Table 4: Comparison of average, minimum, and maximum execution times in microseconds for TSPA and TSPB. Greedy Cycle* is the Greedy Cycle 2-Regret (w=0.5)